

week-01-how-the-web-works

Week 01 — How the Web Works

Session goal: by the end of class you have a working editor, a local server, and your first page open in the browser — plus DevTools habits you will use every week.

The web is a conversation between a browser (client) and a computer somewhere else (server). Today you set up your tools, write your first HTML page, and learn to read Network and Console.

Warm-up (5 min)

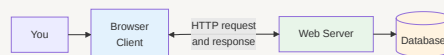
Without installing anything yet, open any website you use daily in Chrome or Edge. Press **F12**. Click the **Network** tab and reload.

Write down (on paper or a note):

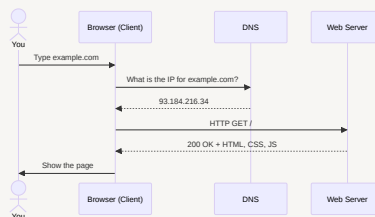
1. How many requests appeared?
2. Can you find one that says **200** ?

Client and Server

Every web app has two roles.



Example — What happens when you open a site



Example Explained

Role	Runs where	This course
Client (frontend)	The user's browser	HTML, CSS, JavaScript, React
Server (backend)	A computer somewhere else	Node.js, Express, MongoDB

When something breaks later, ask first: **client problem or server problem?**

Note: React runs on the client. Express runs on the server. They talk over HTTP.

Class activity A — draw the client server diagram with pen and paper(5 min)

1. Draw **2 boxes**: left = **Client (Browser)**, right = **Server**.
2. Draw an arrow **Client → Server** and write **Request**.
3. Draw an arrow **Server → Client** and write **Response**.
4. (Optional) Draw a small box under Server: **Database**.

Remember: Client = shows page, Server = sends data.

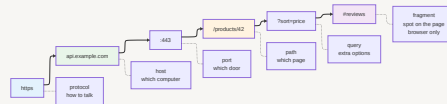
Anatomy of a URL

A URL has named parts. You will read them every day.

Example

`https://api.example.com:443/products/42?sort=price#reviews`

protocol host port path query fragment



Example Explained

Part	Meaning	Notes
<code>https</code>	Protocol	Encrypted HTTP — always use it in production
<code>api.example.com</code>	Host	DNS turns this into an IP
<code>443</code>	Port	Often hidden. 80 = HTTP, 443 = HTTPS, <code>5500</code> / <code>5173</code> = your later local servers
<code>/products/42</code>	Path	Which resource
<code>?sort=price</code>	Query	Extra params (<code>&</code> joins more)
<code>#reviews</code>	Fragment	Browser-only — not sent to the server

Class activity B — URL teardown (8 min)

1. Find any URL on the internet.
2. Copy the complete URL.
3. Split the URL into:
 - **Protocol**
 - **Host**
 - **Port** (if any)
 - **Path**

- Query
- Fragment

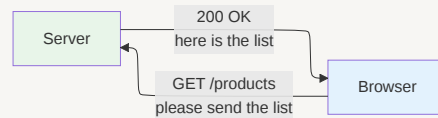
4. Make sure your URL contains **all six parts**. If needed, use a Live Server URL.

5. Write each part separately.

- Change the product or search for something and check which part of the URL changes.

HTTP Methods and Status Codes

HTTP methods say what you want. Status codes say what happened.



Example

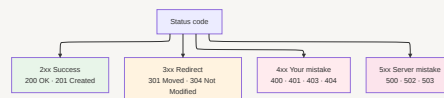
```

GET    /products    → read a list
POST   /signup     → create an account
PUT    /profile    → update a profile
DELETE /todos/3    → remove a to-do
  
```

Example Explained

Method	Meaning	Example
GET	Read	Load a page or product list
POST	Create	Submit a signup form
PUT / PATCH	Update	Edit a profile
DELETE	Remove	Delete a to-do

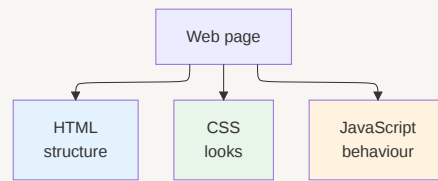
Status codes worth memorising:



Range	Meaning	Examples
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect / cache	301 Moved, 304 Not Modified
4xx	Your request is wrong	400, 401, 403, 404
5xx	The server failed	500, 502, 503

Habit for the whole course: **4xx** → **fix the request**. **5xx** → **fix the server**.

The three web languages



- **HTML** — structure (what exists and what it means)
- **CSS** — presentation (how it looks)
- **JavaScript** — behaviour (what happens on interaction)

Think of a house: HTML = walls and rooms, CSS = paint and furniture, JavaScript = electricity and buttons that *do* something.

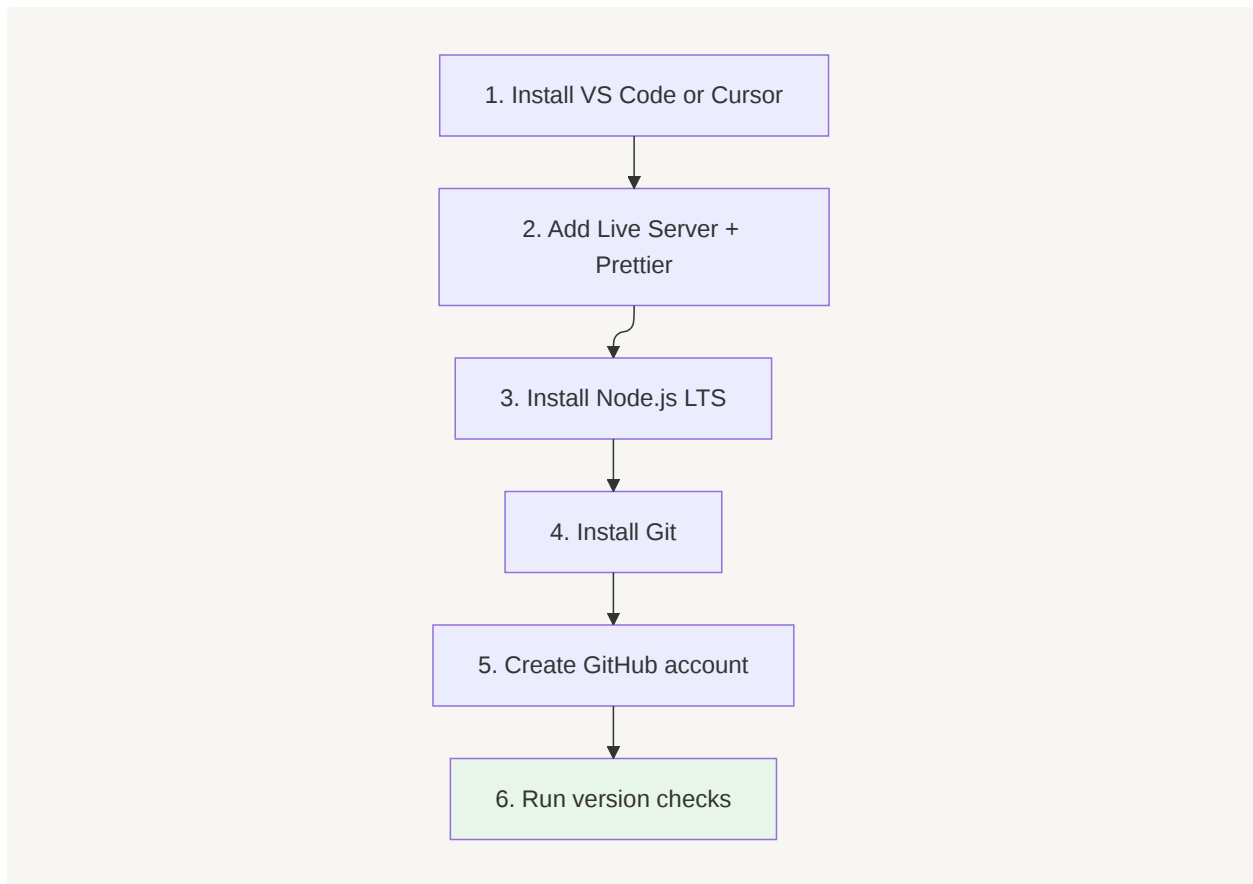
Class activity C — HTTP Status Code Activity (5 min)

1. Open a website and open **Developer Tools** → **Network**.
2. Find **one status code from each family**:
 - **2xx** → Success
 - **3xx** → Redirection
 - **4xx** → Client Error
 - **5xx** → Server Error
3. Take a **screenshot** of each status code from the Network tab.
4. Paste all **4 screenshots**.
5. Write the status code and its meaning below each screenshot.

Done when: You have one screenshot each for **2xx, 3xx, 4xx, and 5xx**.

Environment Setup

Install in order. Do not skip the `--version` checks.



Example — Editor Extensions

Extension	Why
Live Server	Serves HTML with auto-reload
Prettier	Formats on save
ESLint	Warns about JS mistakes (later weeks)
Auto Rename Tag	Renames closing tags with opening tags

Install **VS Code** (code.visualstudio.com) or **Cursor** (cursor.com).

Turn on format-on-save: Settings → search `format on save` → enable.

Example — Verify Node and Git

```
node --version    # v20.x.x or newer
npm --version     # 10.x.x or newer

git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git --version
```

Example Explained

- Download **Node.js LTS** from nodejs.org
- Install Git from git-scm.com
- Create a GitHub account with a professional username
- If a command prints `command not found`, close and reopen the terminal first

Warning: Skipping `--version` checks is how people think they installed something when the terminal cannot find it.

Class activity D — Tools Installation & Setup Activity (15 min)

1. Install **Node.js**, **npm**, and **Git** if they are not already installed.
2. Check the versions:
 - `node -v`
 - `npm -v`
 - `git --version`
3. Make sure **Live Server** is installed in your code editor and working.
4. Make sure **Prettier Formatter** is installed and working.
5. Test Prettier by writing messy code, saving the file, and checking that it formats automatically.
6. Take screenshots showing:
 - Node.js, npm, and Git versions
 - Live Server installed/working
 - Prettier installed/working
7. Paste all the screenshots.

Done when: Node.js, npm, Git, Live Server, and Prettier are all installed and working.

Your First Page

Create a folder `hello-web` and a file `index.html`.



Example

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Hello Web</title>
  </head>
  <body>

```

Result

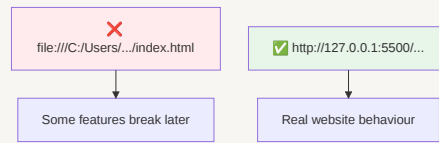
Page:

Address bar should look like:

Example Explained

Part	Meaning
<code><!DOCTYPE html></code>	This document is HTML5
<code><html lang="en"></code>	Root element; language for accessibility
<code>charset="UTF-8"</code>	Correct characters
<code>viewport</code>	Makes the page scale on phones
<code><title></code>	Text in the browser tab
Live Server	Opens <code>http://127.0.0.1:5500/...</code> — a real URL

Important: Do **not** open the file as `file://`. Always use Live Server this course.



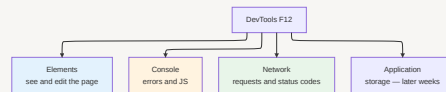
Class activity E — personalise and serve (12 min)

1. Type the skeleton yourself (do not paste)
2. Replace `Your Name` with your name
3. Add a short bio paragraph
4. Add an unordered list of **three things you want to build** this semester
5. Serve with Live Server and click the button

Done when: the address bar shows `http://...` (not `file://`), your name is on the page, and the button alerts.

DevTools

Open with `F12` or `Ctrl+Shift+I` (`Cmd+Option+I` on Mac).

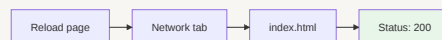


Example

Tab	Use it for
Elements	Live DOM; tweak HTML/CSS to experiment
Console	Errors, <code>console.log</code> , quick JS
Network	Every request, status code, timing
Application	localStorage, cookies, cache (later)

Example Explained

- **Network:** reload. Find `index.html`. Status should be `200`.
- **Console:** type `document.title` and press Enter.
- **Elements:** edits are temporary — reload throws them away.



Class activity F — three DevTools drills (10 min)

On your Live Server page:

1. **Network:** reload. Find `index.html`. Confirm status `200`. Screenshot it.

2. **Console:** Type `document.title` and press Enter. Check the result.
3. **Elements:** double-click the `<h1>` text and change it. Reload — notice the file did **not** keep the DevTools edit.

Done when: you have a Network `200` screenshot, a one-sentence error explanation, and you can say why the Elements edit disappeared.

Lab (in class) — first page + proof of DevTools

Polish `hello-web/index.html` and prove the environment works.

- ☐ Valid basics: doctype, `lang`, charset, viewport, title
- ☐ Your name, bio paragraph, list of three goals
- ☐ Button that alerts on click
- ☐ Served with Live Server (`http://...`, not `file://`)
- ☐ Network screenshot showing `index.html` → `200`
- ☐ One deliberate Console error explained in one sentence

Pass criteria: page served over `http://localhost` (or `127.0.0.1`), screenshot submitted, error explained in your own words.

Submit: `index.html` + Network screenshot + the one-sentence error note.

Homework

Due before Week 02. Submit in the class channel / LMS as listed for each task.

Task 1 — Prove the toolchain

Finish all installs. Paste the exact terminal output of:

```
node -v
npm -v
git --version
```

Submit: a single message or screenshot with all three outputs visible.

Task 2 — Network detective (hands-on)

Pick any website you use daily. Open DevTools → Network, reload, and answer in a short note:

1. How many requests did the page make?
2. What is the largest file, and what type is it (JS, image, CSS...)?
3. Find one request that is **not** `200`. What status is it, and what do you think happened?

Submit: answers + a screenshot of the Network panel showing the non-200 row.

Checklist before Week 02

- ☐ I can explain client vs server in one sentence each

- ☐ I can name what `200`, `404`, and `500` mean
- ☐ I can label the parts of a URL on a real address bar
- ☐ Node, npm, and Git all print a version number
- ☐ I can serve a page with Live Server
- ☐ I can open DevTools and use Console + Network without help